Security Threat Model — Multi-Account Helix OTA

Revision: 1 **Last modified:** 2026-07-10T11:18:54Z

> **Scope.** This is the security THREAT MODEL for adding an ACCOUNT (tenant) layer to the Helix OTA control plane (`server/`, Go/Gin modular monolith). It enumerates the threats the multi-account feature introduces or exposes, maps each to a concrete control, and specifies the anti-bluff proofs the implementation MUST carry. It is **research + planning only** — no code, no source edit. Entity/column/role shapes are **owned by `20_target_multitenancy_data_model.md` (SSOT)**; this doc *consumes* those shapes (cites, never redefines) and adds the security analysis `20_*` explicitly delegated to it — the paired-mutation proof that cross-account reads are impossible (`20_*` §0, §6, Honest boundary).
>
> **Reading order.** Read `00_INDEX.md`, then the authoritative as-is audits `10_existing_auth_and_project_model.md` (auth/identity/project) and `11_existing_upload_and_device_update.md` (upload/device/trust-boundary), then the SSOT `20_*`. Every "as-is" claim here carries a `file:line` those docs established (re-verified against source where load-bearing); every "to-be" control is a recommendation-with-tradeoffs for operator decision, never a silent choice (§11.4.6 / §11.4.66). Sibling `24_project_side_cli.md` (API keys) and `25_device_side_update_client.md` (device tokens) own the credential *mechanisms*; this doc owns their *threat analysis*.
>
> **Threat-model method.** Sections 1-5 are asset-and-vector-driven (the primary tenant-isolation asset first, per §11.4.132 risk-descending order); section 6 is a STRIDE pass over the new token/session surface. Each control names its layer (app-layer scoping / DB RLS / token-claim / config trust-boundary) so defense-in-depth is explicit, not assumed.

---

0. Assets, trust boundaries, and the threat actors

Assets to protect (highest value first):

Asset	Why it is the target	As-is exposure
Cross-account data confidentiality (Device/Artifact/Release/Deployment/Telemetry/Group rows)	The whole point of tenancy; a leak crosses an *organization* boundary, not a user boundary	**Zero isolation today** — these entities carry no `account_id`/`project_id` (`store.go:36-178`; 10_* §4)
Super-admin identity (`is_super_admin`)	"Sees/controls everything" — one credential = total cross-tenant breach	Modeled by `20_*` §6 as a global boolean; bootstrap trust-boundary from config only
Signing trust (which key verifies an artifact)	A forged-key acceptance ships malicious firmware to devices	One global `HELIX_ARTIFACT_PUBKEY`, key never from request (`handlers_artifact.go:274-288`) — the invariant to preserve
Credentials (API keys, device tokens, refresh tokens, token secret)	Grant scoped or unscoped access; leak = impersonation	HMAC token secret has a **predictable dev fallback** (`config.go:180-184`); refresh tokens in-memory single-use (`handlers_auth.go:28-72`)
Artifact bytes at rest / in transit	The payload devices flash	Bytes **validated then discarded**; `StorageRef` is a placeholder (`handlers_artifact.go:184`; 11_* Honest-gap 1)

Trust boundaries (established precedent the account layer MUST inherit). The codebase already refuses to derive trust from a request: the artifact verify key comes ONLY from server config — *"A request-supplied key is NEVER trusted … There is deliberately no request path into this function"* (`resolvePublicKey`, `handlers_artifact.go:274-288`), and `TrustTLSProxy` is an explicit operator boolean, not inferred from `X-`

`Forwarded-Proto` (10_* §7). **The account-tenancy claim MUST follow the identical rule: server-minted, server-verified, never self-asserted by the caller** (25_* §1.2). This is the single load-bearing security principle of the whole feature.

Threat actors: (T-EXT) an unauthenticated external attacker; (T-TENANT) a legitimately-authenticated principal of account A attempting to reach account B's data (the primary multi-tenant actor — OWASP: the boundary crossed is "between organizations rather than between individual users"); (T-DEVICE) a compromised or spoofed device token; (T-SIGNER) an attacker attempting to get a malicious artifact accepted; (T-INSIDER) a compromised super-admin credential.

---

1. Tenant isolation — the PRIMARY threat: cross-account data leakage

This is the highest-risk asset (§0) and therefore analyzed first (§11.4.132). Today there is **no data-level tenant isolation**: any principal with a global `operator`/`viewer` role sees and mutates the entire fleet, because Device/Artifact/Release/Deployment/Telemetry/Group carry no tenant column and their handlers consult no membership (10_* §2, §4; "there is no data-level tenant isolation today"). The multi-account feature's core security job is to close this by construction. Cross-tenant leakage "rarely stems from a single obvious flaw" but from "a missing tenant filter in a query or a background worker running outside tenant context" (OWASP / industry) — so the defense must be layered, not a single filter.

1.1 Leak-vector enumeration (each grounded in an as-is `file:line`)

#	Vector	Concrete as-is path	Class
V1	**Cross-account LIST**	`GET /releases` / `GET /deployments` return the whole fleet; no account filter exists (`server.go:206-210`; `AuditFilter`/`ReleaseFilter` carry no account, `store.go:280-285`)	Broken function-level + object-level authZ
V2	**IDOR / BOLA on GET-by-id**	`GET /artifacts/:artifactId`, `GET /releases/:releaseId`, `GET /deployments/:deploymentId`	**OWASP API1:2023 BOLA** — "user

#	Vector	Concrete as-is path	Class
		resolve an opaque app-minted `TEXT` id with only a global `requireRole` gate — no ownership check (`server.go:198-211`)	is authenticated but not authorized for the specific object”
V3	**Cross-account device update leak**	`GET /client/update` resolves an active deployment by `(dev.OSType, dev.Model, dev.Group)` globally (`handlers_client.go:43`); a device enrolled by account A whose `(os,model,group)` matches would be offered account B’s deployment (25_* §1.1)	Cross-tenant leakage by construction
V4	**Cross-account WRITE / poisoning**	`POST /releases`, `POST /deployments` create fleet-global rows; monotonicity keyed `(os_type,target_model)` and active-uniqueness `(os,model,group)` are global (`store.go:349,359`), so account A’s version bump can block or collide with account B’s (10_* §8)	Shared-resource poisoning / integrity
V5	**Cross-account audit read**	`GET /audit` is admin-only but has no account/project filter (`handlers_audit.go:133-175`; rows carry no `account_id`, `store.go:123-134`); a future per-account admin would read every tenant’s audit trail	Confidentiality of an isolation-monitoring surface
V6	**Namespace collision oracle**	Global-unique project/device-`hardware_id`/group names (`schema_postgres.sql:241,30,102`) leak the existence of another tenant’s names via a uniqueness-conflict error	Enumeration side-channel

1.2 How `20_*`'s model closes each vector (defense-in-depth: app-layer AND DB RLS)

`20_*` (SSOT) fixes **shared-schema + a denormalized `account_id` on every tenant-owned table + PostgreSQL Row-Level Security** as the tenancy model. The security value is that the two layers fail independently — AWS's RLS guidance is that database-enforced RLS "works even when your application code has bugs," and the OWASP Multi-Tenant cheat sheet mandates both a request-lifecycle tenant context AND database RLS. Mapping:

Vector	App-layer control (primary)	DB-layer control (belt-and-suspenders)	**Toke claim cont**
V1 LIST	`*Filter` structs gain `AccountID`/`ProjectID` (`20_*` §3.2); every list query is account-scoped	RLS policy `account_id = current_setting('app.current_account')` filters rows the query forgot to (`20_*` §4 step 8)	list s deriv from verif toke `accou` neve quer para
V2 BOLA/ IDOR	Every get/create/update gains an explicit `accountID` param — "a forgotten scope is a **build error**, not a runtime leak" (`20_*` §3.2 Option A recommendation); `requireAccountAccess` before the resolve	RLS makes a by-id read of another account's row return **zero rows** even if the handler forgot the check	obje `accou` comp to th toke `accou`
V3 device leak	`ActiveDeploymentForTarget` gains `(account_id, project_id)` (`20_*` §2.2 #4; 25_* §1.2) — a device "only ever sees its own account's updates"	RLS on `deployments` scoped by the device's account GUC	devi toke carri serv mint `accou` `proj` (25_*
V4 write/ poison	monotonicity/active-uniqueness keys re-scoped to `(account_id,`	composite FK `(project_id, account_id) → projects` guarantees `resource.account_id ==`	writ acco take

Vector	App-layer control (primary)	DB-layer control (belt-and-suspenders)	Toke claim cont
	`project_id, …)` (`20_*` §2.2 #3, #4)	`project.account_id` at the DB — no drift (`20_*` §2)	toke scop neve body
V5 audit	`AuditEntry` gains `account_id`/`project_id` + an account filter on the read path (`20_*` §2.1)	RLS on `audit_logs`; a per-account admin sees only their tenant	audi scop toke acco (sup adm exen §2)
V6 collision	uniqueness re-scoped per account: `UNIQUE (account_id, name)` etc. (`20_*` §2.2 #1,#6,#2) removes the cross-tenant conflict oracle	—	—

Recommendation (security-critical, echoing `20_*` §3.2): choose the **explicit `accountID` parameter** (compile-time-enforced) over context-implicit scope for the single-entity paths, *because* the entire feature is tenant isolation and a compile- time guarantee beats a runtime one — a `context.Context`-carried scope silently reads unscoped when the context is missing it, which is exactly the "background worker outside tenant context" leak. Keep RLS as the independent second layer regardless. **Tradeoff:** larger diff (every OTA handler + call site touched) — accepted; the alternative's convenience is not worth a silent-leak class.

1.3 Mandatory anti-bluff proof (this doc's delegated obligation, `20_*` §0/§6)

Isolation is only as strong as its proof. The implementation MUST carry, per §1.1/§1.2 and §11.4.27 (no fakes beyond unit tests) + §11.4.69 (captured evidence):

- **Negative isolation test (integration, real server + real store).** A principal scoped to account A issues each of V1-V5's requests against account B's ids and MUST receive `403`/empty — captured

request+response evidence under `docs/qa/<run-id>/`. A cross-account read that returns data is a release blocker.
- **Paired §1.1 meta-test mutation (the analyzer cannot bluff).** Mutate the scope check (drop the `WHERE account_id = …` / disable the RLS policy / return true from `requireAccountAccess`) and assert the isolation test **FAILs**. A mutation that leaves the test green means the test never exercised the boundary — itself a §11.4 bluff (§11.4.107(10) self-validated analyzer: golden-good scoped session PASS + golden-bad cross-account session FAIL, wired into the meta-test).
- **DB-layer proof independent of the app.** With the app-role session variable set to account A, a raw `SELECT * FROM devices` MUST return zero of account B's rows even when the SQL carries a forged/absent `WHERE` — proving RLS holds when application code is wrong (the AWS "works even when your application code has bugs" claim, verified not assumed). Requires the app to connect as a **non-owner, non-`BYPASSRLS`** role (`20_*` §4 step 8) — see §2.

---

2. Super-admin blast radius — the highest-value single credential

The super-admin ("sees/controls everything", `00_INDEX` §1.4; modeled as the global `users.is_super_admin` boolean, `20_*` §1.2/§6) is the maximum-blast-radius asset: one compromised super-admin credential is a **total cross-tenant breach**. The threat model for it is not "can it be misused" (it is all-powerful by design) but "is every use **attributable, revocable, and impossible to obtain by self-assertion**."

2.1 Bootstrap trust boundary (Spoofing / Elevation-of-privilege)

Threat: a request elevates itself to super-admin. **Control:** `is_super_admin` is settable **only via config/env bootstrap, never a request** — the identical rule the codebase already enforces for the token secret (`config.go:180-184`), the admin bootstrap (`main.go:96-104`: `HELIX_ADMIN_PASSWORD` → a single static user, unset ⇒ no user), the TLS-proxy trust flag, and `resolvePublicKey` (`handlers_artifact.go:274-288`). The first super-admin is seeded from a `HELIX_*` env field (`20_*` §6; 10_* §7). **Anti-bluff proof:** a test that POSTs `is_super_admin=true` in every

account/user/ membership write body and asserts it is ignored (the flag is never request-writable); paired mutation makes the field request-writable and asserts the test FAILs.

2.2 No `BYPASSRLS` — policy-based bypass only (Repudiation / Auditability)

Threat: super-admin access that is invisible to the database and therefore unloggable/unrevocable per query. **Control (from `20_*` §6, security-load-bearing):** the app connects as a **non-owner, non-`BYPASSRLS`** role and a super-admin request sets an additional GUC (`app.is_super_admin = 'on'`) that each policy's `USING` clause also accepts — NOT a Postgres `BYPASSRLS`/superuser role (which *always* bypasses RLS and is invisible to policies). AWS explicitly recommends "policy-based admin access over privilege-based bypass" because it "keeps admin access **auditable and revocable**." **Why it matters here:** a `BYPASSRLS` role cannot be scoped or logged per query, so a super-admin's cross-tenant reads would be un-attributable — the exact repudiation risk. The OWASP Multi-Tenant cheat sheet lists *"Skip tenant validation for 'internal services'"* as an explicit **Don't** — the policy-bypass GUC is the compliant inverse (the bypass is a visible, revocable policy branch, not an un-checked path).

2.3 Every super-admin action logged with the affected tenant (Auditability)

Threat: a super-admin action that does not record which tenant it touched is a forensic dead end. **Control:** every audit row for a super-admin action MUST carry the **affected `account_id`** (`20_*` §2.1; audit today carries none, 10_* §6). Populate the long-empty `AuditEntry.UserID` (always empty today, `audit_wire.go:41-43`) with the real `user_id` so the actor is a durable identity, not a bare subject string. Extend the audit middleware to record reads *when the actor is super-admin* (normal reads are not audited today, `handlers_audit.go:21-50`) — a super-admin cross-tenant read is exactly the event a tenant needs in its audit trail. **Impersonation:** if `21_*`/`23_*` adds a "super-admin acts as account X" impersonation flow, every impersonated action MUST log BOTH the real super-admin `user_id` AND the impersonated `account_id`/subject (dual attribution) — an impersonation that logs only the impersonated identity is a repudiation hole.

2.4 Revocability (the anti-bluff proof `20_*` §6 delegates here)

Threat: a super-admin bypass that survives revocation. **Proof (MUST carry):** a paired test showing (a) a non-super-admin session with `app.current_account = A` cannot read account B's rows even with a forged `WHERE` (the §1.3 DB-layer proof), and (b) clearing `is_super_admin` (and/or the `app.is_super_admin` GUC) **immediately** re-scopes the session — the next query sees only the actor's own account rows. A super-admin bypass that persists after the flag/GUC is cleared is a §11.4 isolation-layer bluff and a release blocker. **Operational hardening (recommendation, tradeoffs):** keep super-admins few and named; consider requiring a second factor / step-up for destructive super-admin actions (delete-account, cross-tenant key rotation) — *tradeoff:* step-up adds friction to a break-glass role; at minimum every super-admin action is audited per §2.3 so misuse is detectable after the fact.

---

3. Credential handling (§11.4.10 / §11.4.10.A)

Two new long-lived credential classes are introduced — account+project-scoped **API keys** (mechanism owned by 24_* §1) and account/project-scoped **device tokens** (mechanism owned by 25_* §1) — alongside the existing HMAC access/refresh tokens. §11.4.10 (no leak) and §11.4.10.A (pre-store leak audit) bind all of them.

3.1 Hash-at-rest, cleartext-once (Information disclosure)

Threat: a stored credential is read from the DB / a backup / a log and replayed. **Control:** never store a credential in cleartext. The `api_keys` sketch already stores only `key_hash` (`001_initial_schema.up.sql:60`; 24_* §1.1) — the cleartext key (recommended opaque `helixk_<account-short>_<random>`) is shown **once** at creation and never persisted in cleartext. This matches the surveyed industry norm — 24_*'s negative finding (§11.4.99):* none* of Mender/balena/Expo/npm stores the cleartext token server-side, all store a hash and show the secret once. **Recommendation:** hash API keys and refresh tokens with a slow/keyed function (argon2id or HMAC-SHA256 with a server key) rather than a bare digest, so a DB read cannot brute-force short keys; user passwords use the existing `password_hash` column (`20_*` §1.2), never a fast hash. *Tradeoff:* argon2 costs CPU on the (rare) key-create path — negligible vs. the offline-crack risk.

3.2 Scope is authoritative from the stored row, never self-asserted (Elevation)

Threat: a client presents a key and claims a broader `(account, project)` scope than the key grants. **Control (the §0 trust boundary applied to credentials):** the server derives `(account_id, project_id, permissions)` **from the stored key row**, never from a request field (24_* §1.1) — a client "cannot upload into an account its key does not grant" (24_* §3). This is the same rule as `resolvePublicKey`/`TrustTLSProxy`. **Recommendation:** prefer 24_*'s **Option B token-exchange** — *the long-lived key touches the wire once per session and is exchanged for a short-lived scoped access token — so the standing on-wire exposure of the powerful credential is minimized (§11.4.10).* Tradeoff:* one extra round trip + a `21_*` tenant-claim dependency; Option A (direct-key Bearer) is an acceptable revocation-immediate interim.

3.3 Revocation and expiry (the kill switch)

Threat: a leaked key stays valid forever. **Control:** every credential is revocable and expirable — `api_keys` carries `revoked_at` + `expires_at` (`001_initial_schema.up.sql: 57-71`; 24_* §1.1); refresh tokens are TTL-bounded single-use-rotating already (`handlers_auth.go:28-72`). **Revocation must be immediate at the *check* layer:** Option A (per-request row lookup) revokes instantly; Option B's short-lived token cannot be retroactively killed but auto-expires within its TTL (15 min default, `config.go:54-55`) — a stated, bounded residual (24_* §1.3). Device tokens are the weakest link: 24 h default TTL (`config.go:56-57`) is a long replay window — **recommendation:** shorten device-token TTL and lean on refresh, and make device tokens per-device + individually revocable so a single compromised device does not require a fleet-wide key rotation (25_* §1.3 Option B bootstrap→rotate gives per-device revocable credentials at scale).

3.4 No-leak enforcement (§11.4.10) and pre-store leak audit (§11.4.10.A)

Threats & controls:

- **In-transit/log leak.** Credentials MUST never be printed or logged — the CLI redacts to `helixk_…<last4>` in any diagnostic (24_* §1.2). Server logs MUST never echo the presented bearer/key. **Proof:** a unit test asserting the raw token never appears in any emitted log line (24_* §6); paired mutation logs the raw token and asserts FAIL.

- **At-rest-in-git leak.** `.env/*credentials/.helix-ota/` are gitignored project-wide (§11.4.10/§11.4.30); the CLI ships a `.gitignore` fragment + `.env.example` (§11.4.77) and a `helix-ota doctor` audits for an accidentally-tracked token (24_* §1.2). File perms `chmod 600` on the credential file, `chmod 700` on its parent (§11.4.10).
- **Pre-store leak audit (§11.4.10.A).** When an operator supplies any secret to be stored (a signing key, an API key, the token secret), the storing agent MUST FIRST audit the repo for a prior leak of *that value* — `git ls-files | xargs grep -l <value>` (tree) and `git log -S<value> --all` (history) — surface findings before storing, and on a hit open a §6/§7 incident + redact + record rotation-required. This gates the super-admin bootstrap secret and any per-account signing key (§4) before it lands.
- **The predictable-dev-secret trap (as-is finding, high severity).** The HMAC token secret falls back to the literal `"helix-ota-dev-token-secret-change-me"` when `HELIX_TOKEN_SECRET` is unset (`config.go:180-184`; 10_* §1). Under multi-account this is catastrophic: a predictable signing key lets an attacker mint a token with **any** `account_id` claim and defeat tenant isolation entirely (§6 Spoofing). **Recommendation:** in a multi-account build, refuse to start (fail-fast, like the config brick's other malformed-value paths, `config.go:118-196`) when `HELIX_TOKEN_SECRET` is unset — the dev fallback is acceptable single-tenant but is a tenant-isolation break multi-tenant. *Tradeoff:* loses one-command local run; mitigate with a documented `.env.example` dev secret, never a code default.

---

4. Artifact-signature trust boundary — per-account signing without breaking "key never from request"

The invariant to preserve. Today `resolvePublicKey` returns one global ed25519 `HELIX_ARTIFACT_PUBKEY` and takes the verify key **only** from `s.pubKey` — the source comment is explicit: *"the verification key MUST come exclusively from server configuration … A request-supplied key is NEVER trusted — accepting one would let an attacker sign a malicious artifact with their own key and present that key, defeating signature verification entirely … There is deliberately no request path into this function"* (`handlers_artifact.go:274-288`; key loaded from config `config.go:187-192`, wired `server.go:109-112`). This is the T-SIGNER defense and it MUST survive the account layer.

4.1 Threat: one global key means any account's signer signs for the platform

With a single shared key, whoever holds the signing private key can sign an artifact that verifies for **every** account — there is no per-tenant signing boundary (24_* §3; 25_* §5). Two sub-threats: (a) a compromised global signing key freezes/poisons the *entire* fleet (the "compromised signing key that can't be rotated can freeze your entire fleet" risk); (b) a rogue tenant-signer can publish into another tenant.

4.2 Control: a per-account key REGISTRY, keyed on the resolved `account_id`

Design (recommendation, from 24_* §3 / 25_* §5). Extend the *lookup* — not the trust source — to a per-account server-side key registry: `resolvePublicKey` becomes `resolvePublicKey(accountID)` returning the account's ed25519 verify key from a config-/KMS-backed registry, with the global key as a fallback for un-migrated accounts. **The trust boundary is unchanged:** the verify key still comes ONLY from server config/registry, never from the request — **only the lookup *key* (the `account_id`) gains an account dimension**, and that `account_id` is itself the server-verified token claim (§6), not a request field. The device side mirrors this: `VerifyBeforeApply` checks against **its account's** public key, provisioned at enrollment **from server config, never from the update offer** (25_* §5.5) — so a malicious offer cannot smuggle its own key onto the device either. *Tradeoff:* a registry lookup + per-account key lifecycle vs. one global key; a shared global key is unacceptable long-term for tenant isolation, so per-account is the target, staged behind the global fallback during migration.

4.3 Per-account key rotation and storage (key-management threats)

Key management is "the operational challenge most teams underestimate … planned before you ship, not after a key is leaked." Recommendations grounded in the surveyed standards:

- **Rotation.** Support per-account key rotation with an overlap window (old key still verifies previously-signed artifacts until they age out; new key signs new artifacts). NIST crypto-period guidance is 1-3 years for signing keys — the registry MUST allow rotation without a fleet freeze. **Key never from request** holds across

rotation: the device learns the *new* key from server config/enrollment, never from the artifact.
- **Compromise resilience (aspirational, flag for 21_*/ops).** TUF's model — threshold (M-of-N) signatures + separation of signing duties + offline keys for the most sensitive role — is the mature target for a fleet updater; adopting even a subset (e.g. an offline root that certifies per-account online signing keys) bounds the blast radius of an online key compromise to one account. *Tradeoff:* full TUF is a large lift; the near-term win is per-account key separation + a rotation path, with TUF-style thresholds noted as the hardening horizon (§11.4.112 — not claimed as shipped).
- **Storage.** Per-account private signing keys live in a KMS / secret store (Azure Key Vault multitenant guidance: "use a separate Key Vault for each tenant"), never in git, gated by the §11.4.10.A pre-store audit. The server holds only *public* verify keys in its registry.

4.4 Anti-bluff proof

An artifact signed by account A's key MUST verify for account A and **fail** for account B (negative cross-account signing test); a request that attempts to supply its own verify key MUST be structurally impossible to honor (no request path — asserted by test + paired mutation that adds a request-key path and proves the isolation/verify test FAILs). Captured evidence: the `Verified` verdict + the account the key resolved from.

---

5. Object-storage gap security (the load-bearing production dependency)

As-is (real gap, not a nicety). `handleUploadArtifact` validates the uploaded bytes (S1-S6) then **discards them**; `StorageRef` is a synthesized placeholder `s3://helix-artifacts/<id>` (`handlers_artifact.go:184`) and the device download URL is `ArtifactBaseURL/<id>.zip` (`handlers_client.go:232-234`) pointing at an external Storage brick **not present in-repo** (11_* Honest-gap 1; 24_* §4). So there is no artifact-byte confidentiality/integrity boundary to attack *yet* — but the moment real object storage lands, it becomes a first-class tenant boundary. This section specifies its security requirements so the storage seam is designed isolated, not retrofitted.

Threats when real storage lands, and controls:

Threat	Control (recommendation)
Cross-account byte access (tenant A reads tenant B's payload)	**Per-account bucket or key-prefix isolation** — `s3://…/<account_id>/<project_id>/<artifact_id>` (or a bucket-per-account for high-compliance tenants), so tenant A's bytes are physically namespaced from tenant B's (24_* §4.2; 25_* §5.6)
Unauthenticated / over-broad download URL	**Signed, time-bounded download URLs** minted per request, scoped to the device's account (never a public or long-lived URL); the URL grants read of exactly one artifact for a short window
Handler-embedded storage client drift	A **`Storage` interface seam** (like `store.Repository`) — dev uses rootless-podman **MinIO** (§11.4.161 via the containers submodule), prod uses S3/GCS — no ad-hoc client in handlers (24_* §4 #3)
Integrity at rest / on serve	The device re-verifies SHA-256 + signature before apply regardless of transport (`VerifyBeforeApply`, 11_* §3) — storage compromise cannot inject a payload that passes verify, but storage MUST still be integrity-checked so a corrupted byte-range fails closed, not silently
IDOR on the download path	The download URL's `account_id`/`project_id` is derived from the verified device token, never a URL/query param (the §0 trust boundary applied to storage)

Honest dependency (§11.4.6). Until real per-account object storage + signed URLs land, an end-to-end "device downloads and applies the exact uploaded bytes" cannot be proven — the CLI/device e2e layers are honestly `SKIP-with-reason: object_storage_absent`, never a fake

PASS over discarded bytes (24_* §6; 25_* §5 #6). The storage seam is a prerequisite milestone `30_delivery_plan.md` sequences before any e2e isolation claim.

---

6. Token / session threats — STRIDE pass over the new surface

The new surface is the tenant-aware token: `Claims{sub, roles, iat, exp}` today (`token.go:31-36`, re-verified — no `nbf`, no `jti`, no tenant field) gains a server-minted `account_id`/`project_id` claim (`20_*` §2.2 #7; 24*/25_* consume it). The token is an HMAC-SHA256 two-part opaque blob verified with constant-time `hmac.Equal` (10_* §1). STRIDE:

STRIDE	Threat on the new surface	As-is fact	Control (recommendation)
Spoofing	**Account-claim forgery** — a caller mints/edits a token carrying a foreign `account_id`	HMAC signature over the payload defeats naive edits (verify `token.go:80-89`) — BUT the **predictable dev-fallback secret** (`config.go:180-184`) lets an attacker who knows it forge ANY claim	Fail-fast on unset `HELIX_TOKEN_SECRET` in a multi-account build (§3.4); rotate the secret if ever exposed (§11.4.10.A). The claim is server-minted only (§0) — no login path lets the caller choose its `account_id`
Tampering	Flip `account_id`/`roles` in the payload	constant-time `hmac.Equal` verify rejects a tampered payload under a secret the attacker lacks	Same as Spoofing — secret secrecy is the whole defense; there is no per-claim signature, so the token secret is the crown jewel
Repudiation	An action cannot be attributed to a real identity	`AuditEntry.UserID` is always empty (`audit_wire.go:41-43`); audit rows carry no account	Populate `UserID` from the resolved `user_id` + `account_id` on the audit row (§2.3); super-admin reads audited

STRIDE	Threat on the new surface	As-is fact	Control (recommendation)
Info disclosure	Token leaks (log, referer, storage) → impersonation within its scope	refresh tokens in-memory single-use-rotating (`handlers_auth.go:28-72`); access TTL 15 min, device TTL 24 h	Redact tokens in logs (§3.4); short TTLs bound the window; scoped tokens limit blast radius to one account
DoS	Token-verify or key-lookup as an amplification target	verify is cheap HMAC; a per-request key/DB lookup (Option A) adds hot-path cost (24_* §1.3)	Prefer Option B (verify a short-lived token on the hot path, key-lookup once per session); rate-limit login/ exchange
EoP	**Legacy token without an `account_id` claim** treated as unscoped/ global	tokens minted before the claim existed, and OQ-4 legacy rows with NULL `account_id` (`20_*` §5)	**Reject legacy tokens after the account cutover** (a token with no `account_id` claim is denied on scoped routes, not treated as "all accounts"); NULL-`account_id` rows are **visible only to super-admin** during any Option-B migration window (`20_*` §5). A "no claim ⇒ global access" default is the exact elevation to forbid

Replay (cross-cutting). The token has no `jti`/nonce and no `nbf` (re-verified, `token.go:31-36`), so replay within the token's validity window is possible if a bearer is captured (bearer-over-TLS is the transport assumption). Mitigations already partly present: access tokens are short-lived (15 min) and refresh tokens are **single-use-rotating** — a replayed refresh token is detected on second use (`handlers_auth.go:28-72`). **Recommendation:** shorten device-token TTL (§3.3); consider adding a `jti` + a server-side denylist for the

highest-value tokens (super-admin, per-account signing-key operations) so an individual token can be revoked before expiry — *tradeoff:* a denylist adds a per-request check and state; scope it to high-value tokens only, not the whole fleet.

Account-switching (session correctness, not a classic STRIDE cell but security-relevant). `20_*` §2.2 #7 recommends resolving membership per-request so a user in N accounts can switch without re-minting; the security requirement is that the *active* `account_id` in the token/GUC is unambiguous per request — a request must never be evaluated under one account's scope while carrying another's data. `26_*`'s isolation proof (§1.3) must include a switch test: after switching from A to B, a request MUST NOT read A's rows.

Sources verified 2026-07-10

External research per §11.4.8 / §11.4.99. These are security-authority sources, deliberately **distinct** from `20_*`'s data-model sources (Microsoft/WorkOS/dasroot) except where the AWS RLS guidance is the shared control basis; the AWS policy-based-bypass point is re-cited here for its auditability/revocability claim specifically.

- **OWASP API Security Top 10 — API1:2023 Broken Object Level Authorization (BOLA)** — the canonical statement that BOLA (= IDOR at the API layer) is the most prevalent API risk, and that in multi-tenant systems the crossed boundary is "between organizations rather than between individual users"; prevention = authorize every object access against the caller's scope, never trust a client-supplied id: https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/
- **OWASP Multi-Tenant Security Cheat Sheet** — defense-in-depth (app-layer tenant context in middleware/interceptor bound to the authenticated session AND database RLS); *"Get tenant from verified JWT claims — NOT from headers"*; *"Never trust client-supplied tenant IDs"*; IDOR + shared-resource-poisoning as primary vectors; *"Include tenant context in all log entries"* + tenant-isolated audit trails; *"Skip tenant validation for 'internal services'"* listed as an explicit Don't: https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html
- **AWS — Multi-tenant data isolation with PostgreSQL Row-Level Security** — RLS "works even when your application code has bugs" (the defense-in-depth basis for §1.2), the app-role must be non-owner + non-`BYPASSRLS`, and *"prefer policy-based admin access over*

privilege-based bypass … keeps admin access auditable and revocable" (the §2.2 super-admin control): https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/

- **NIST — Security Considerations for Code Signing (NIST Cybersecurity White Paper, 2018)** + **NIST SP 800-57 key-management crypto-periods** — signing-key protection, rotation, and the 1-3-year signing-key crypto-period basis for §4.3: https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.01262018.pdf
- **TUF — The Update Framework, Security** — compromise-resilient software-update signing: threshold (M-of-N) signatures, separation of signing duties, offline keys for the most sensitive role, and key rotation as first-class (the §4.3 hardening horizon); plus Microsoft Azure Key Vault multitenant guidance ("a separate Key Vault for each tenant") for §4.3/§5 per-tenant key storage: https://theupdateframework.io/docs/security/ · https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/service/key-vault

Negative finding (§11.4.99): no surveyed source contradicts the design. None stores a cleartext credential server-side (all hash + show-once — corroborates §3.1); all locate the tenant identity in a server-verified claim, never a client header (corroborates §0/§6); AWS/OWASP both mandate the app-layer + DB-layer *pair*, none treats RLS as sufficient alone (corroborates §1.2 defense-in-depth). The per-account signing-key registry and the composite-FK tenant integrity are Helix-specific applications of these principles — no source prescribes them verbatim ("original work" applied to this schema, consistent with `20_*` §Sources).

Honest boundary (§11.4.6)

- **This is a threat model, not an implementation, and not a completion claim.** No control, RLS policy, key registry, storage seam, or test described here exists in code yet. Every "as-is" fact carries a re-verified `file:line`; every "to-be" control is a recommendation with tradeoffs for operator decision, never a silent choice (§11.4.66). Nothing here is a §11.4 PASS-bluff — it specifies the proofs the implementation must later produce, it does not assert they pass.
- **`20_*` is the SSOT for entity/role shapes; this doc never redefines them.** Where an `account_id`/`project_id` column, the `is_super_admin` flag, the RLS policy form, the `CreateAccountWithOwner` seam, or the OQ-4 migration path is load-bearing here it is *cited* to `20_*` and this

doc reconciles to `20_*` if `20_*` changes (§11.4.186 anti-divergence). The API- key mechanism is 24_*'s; the device-token/enrollment mechanism is 25_*'s; this doc owns their *threat analysis*, not their shape.

- **What the model proves vs. does not.** It proves a *design* under which cross-account reads, claim forgery, credential leak, forged-artifact acceptance, and cross-tenant byte access are each closed by a named control + a falsifiable anti-bluff proof. It does **not** prove those controls are correctly implemented (that is the paired-mutation test suite's job, §1.3/§2.4), nor that shared-schema+RLS is a *guarantee* of isolation — isolation is only ever as strong as the policies + the scope-resolution code, which is precisely why every §1/§2 control ships with the DB-independent-of-app proof and the mutation that must make the test FAIL.
- **Named residual risks carried forward (not solved by this design):** (1) the predictable HMAC **dev-fallback secret** (`config.go:180-184`) is a tenant-isolation break in a multi-account build until fail-fast lands (§3.4 / §6 Spoofing) — the single highest-severity as-is finding;
    1. **real object storage** is absent — the §5 storage boundary is designed but unbuilt, so device-byte e2e isolation is honestly un-provable today (§5 / 11_* Honest-gap 1); (3) the **tenant-aware token claim** and **per-account key registry** are `21_*`/`20_*` dependencies not owned here; (4) the **rollout subsystem** (`internal/rollout/`) tenancy is marked `UNCONFIRMED:` by 10_* — its store must be audited before it is scoped, and until then it is a potential unscoped path this model cannot certify. (5) Replay defenses beyond short TTL + refresh-rotation (a `jti` denylist) are a recommendation, not a shipped control.